

SAGE: Synthesis and Verification of Information-Flow Guards for Tool-Using Agents under Partial Observation: Supplementary Material

ANONYMOUS AUTHOR(S)

This anonymous supplement provides detailed proofs, the concrete execution soundness premise, the PDR feasibility study, and full pseudocode. The main paper is self-contained; these materials provide additional auditability and reproducibility detail.

1 Guard-Trigger Semantics

For $u = (a, t, p)$, a guard clause $c = (a_c, t_c, pf_c, C_c)$ triggers exactly when

$$\text{Trig}(c, x, u) \Leftrightarrow a = a_c \wedge t = t_c \wedge \text{Match}(p, pf_c) \wedge \eta_u(x) \cap C_c \neq \emptyset, \quad (1)$$

$$\text{Deny}_C(x, u) \Leftrightarrow \exists c \in C : \text{Trig}(c, x, u). \quad (2)$$

Thus $K_C(x, u) = \text{deny}$ exactly when $\text{Deny}_C(x, u)$ holds, and it allows otherwise. This is the trigger semantics used by DIRECTSAGE and the verifier.

2 Proofs of Main Results

Numbering note. The theorem and corollary numbers in the main paper are authoritative. Restated results below deliberately carry no supplementary-material serial number; their titles identify the matching main-paper result. The auxiliary results are not restatements.

Proof structure. The safety chain is Lemma 1 $\Rightarrow$ DirectSAGE compilation $\Rightarrow$ Theorem 1 $\Rightarrow$ Corollary 1. Separately, Theorem 2, Corollary 2, and Proposition 1 characterize permissiveness and collision under a declared observation; the remaining results address template realizability and additive repair.

Lemma (Restatement of Lemma 1: One-step danger). *For every safe state x and enabled action u ,*

$$\neg \text{Safe}_B(F_u(x)) \Leftrightarrow \eta_u(x) \cap Q_B(u) \neq \emptyset.$$

PROOF. ($\Rightarrow$) Assume $\neg \text{Safe}_B(F_u(x))$. By definition of Safe_B , there exists $v \in V$ with $F_u(x)(v) \cap B(v) \neq \emptyset$. For $v \notin \text{Wr}(u)$, $F_u(x)(v) = x(v)$, and since x is safe, $x(v) \cap B(v) = \emptyset$, a contradiction. So $v \in \text{Wr}(u)$, and $F_u(x)(v) = x(v) \cup \eta_u(x)$. Since $x(v) \cap B(v) = \emptyset$ (safety of x), the non-empty intersection must come from $\eta_u(x)$: $\eta_u(x) \cap B(v) \neq \emptyset$. Since $v \in \text{Wr}(u)$, $B(v) \subseteq Q_B(u)$, so $\eta_u(x) \cap Q_B(u) \neq \emptyset$.

($\Leftarrow$) Assume $\eta_u(x) \cap Q_B(u) \neq \emptyset$ and choose ℓ in this intersection. By definition of Q_B , there exists $v \in \text{Wr}(u)$ with $\ell \in B(v)$. By Definition 3, $F_u(x)(v) = x(v) \cup \eta_u(x) \supseteq \eta_u(x) \ni \ell$. So $\ell \in F_u(x)(v) \cap B(v)$, giving $\neg \text{Safe}_B(F_u(x))$. $\square$

Theorem (Restatement of Theorem 1: Trace safety). *Assume (i) $\text{Safe}_B(x_0)$ and (ii) a guard set C_B such that, for every safe state x and enabled action u , $\text{Deny}_{C_B}(x, u) \Leftrightarrow \text{Deny}_B^*(x, u)$. Then $\text{TraceSafe}(K_{C_B}, x_0, B)$.*

PROOF. The conclusion concerns every prefix of every controlled trace, so we prove the stronger inductive invariant that every state on an arbitrary finite controlled trace is safe. The inductive step separates the two possible authorization outcomes: a dangerous action is stopped, while an allowed action must have a safe successor by Lemma 1.

Let $\rho = x_0 \xrightarrow{u_0} x_1 \cdots \xrightarrow{u_{n-1}} x_n$ be any finite controlled trace. We show $\text{Safe}_B(x_i)$ for every i by induction on its prefix length.

Base. $\text{Safe}_B(x_0)$ by assumption (i).

Inductive step. Assume $\text{Safe}_B(x_i)$. Consider any enabled action u_i at x_i .

- If $\text{Deny}_B^*(x_i, u_i)$: by assumption (ii), $\text{Deny}_{C_B}(x_i, u_i)$, so $K_{C_B}(x_i, u_i) = \text{deny}$ and no controlled transition exists. The trace cannot proceed through u_i .
- If $\neg \text{Deny}_B^*(x_i, u_i)$: by assumption (ii), $\neg \text{Deny}_{C_B}(x_i, u_i)$, so the guard allows. By Lemma 1 (contrapositive of $\Rightarrow$): $\text{Safe}_B(F_{u_i}(x_i))$. The controlled successor is $x_{i+1} = F_{u_i}(x_i)$, which is safe.

Thus the invariant holds for every trace prefix. Since ρ was arbitrary, every controlled reachable state is safe, which is $\text{TraceSafe}(K_{C_B}, x_0, B)$. $\square$

Corollary (Restatement of Corollary 1: Deployment soundness). *Assume the premises of Theorem 1 and additionally that (i) $s_0 \sqsubseteq_I x_0$; (ii) concrete tools satisfy forward simulation; (iii) complete pre-commit mediation; (iv) atomic authorization and modeled commit, with physical execution preserving the pre-state required by simulation. Then for every concrete trace allowed by K_{C_B} , $I(s_i, v) \cap B(v) = \emptyset$ for all i, v .*

PROOF. The abstract trace theorem alone does not speak about concrete effects. We therefore prove a coupled invariant: each concrete state is overapproximated by the corresponding abstract state, and that abstract state is safe. Forward simulation preserves the first part after an allowed action; Theorem 1 supplies the second.

We show $s_i \sqsubseteq_I x_i$ and $\text{Safe}_B(x_i)$ for every step i of an arbitrary concrete trace by induction.

Base. $s_0 \sqsubseteq_I x_0$ by premise, and $\text{Safe}_B(x_0)$ by the inherited premise of Theorem 1. For any v : $I(s_0, v) \subseteq x_0(v)$ (from $s_0 \sqsubseteq_I x_0$) and $x_0(v) \cap B(v) = \emptyset$ (from $\text{Safe}_B(x_0)$), so $I(s_0, v) \cap B(v) = \emptyset$.

Inductive step. Assume $s_i \sqsubseteq_I x_i$ and $\text{Safe}_B(x_i)$. If the guard denies u_i at x_i , the concrete effect is suppressed by complete mediation; no successor exists. If the guard allows, exactness and Lemma 1 imply $\text{Safe}_B(F_{u_i}(x_i))$, and $x_{i+1} = F_{u_i}(x_i)$. By forward simulation: $s_{i+1} \sqsubseteq_I F_{u_i}(x_i) = x_{i+1}$. Since $\text{Safe}_B(x_{i+1})$: $\forall v : x_{i+1}(v) \cap B(v) = \emptyset$. Since $I(s_{i+1}, v) \subseteq x_{i+1}(v)$: $I(s_{i+1}, v) \cap B(v) \subseteq x_{i+1}(v) \cap B(v) = \emptyset$.

The coupled invariant therefore holds at every concrete step, and its overapproximation component transfers abstract safety to the claimed concrete forbidden-influence safety condition. $\square$

Theorem (Restatement of Theorem 2: Maximal permissiveness). *The theorem has two information levels. With full event observation, the exact guard that denies precisely D is pointwise maximal among deterministic memoryless one-step-safe controllers. Under any fixed runtime observation O , K_O^* is event-wise safe and is maximal among deterministic memoryless controllers measurable under O : for every such event-wise-safe K_O , $K_O(e) = \text{allow}$ implies $K_O^*(e) = \text{allow}$.*

PROOF. Both statements compare allowed-event sets. We first establish the ideal controller that can observe the full event, then show how restricting the available observation changes that ideal.

Full-event observation. The exact guard denies every $e \in D$ by definition and is therefore one-step safe. Let K be any deterministic memoryless one-step-safe controller, and let $e = (x, u)$ be a safe enabled event that K allows. By one-step safety,

$$K(x, u) = \text{allow} \implies \text{Safe}_B(F_u(x)).$$

Since D contains exactly the safe enabled events whose successor violates Safe_B , this gives $e \notin D$. The exact event guard is defined by

$$K^*(x, u) = \text{deny} \iff (x, u) \in D.$$

Thus $e \notin D$ implies $K^\star(x, u) = \text{allow}$, and hence

$$K(x, u) = \text{allow} \implies K^\star(x, u) = \text{allow}$$

for every safe enabled event. Thus the exact guard's allowed-event set contains that of every one-step-safe controller.

Observation-constrained controllers. Now fix O . The candidate $K_O^\star$ is in the required controller class: if $O(e_1) = O(e_2)$, then either both observations belong to $O(D)$ or neither does, so $K_O^\star(e_1) = K_O^\star(e_2)$. It is safe because every $e \in D$ satisfies $O(e) \in O(D)$ and is denied.

It remains to show maximality within this restricted class. Let K_O be any event-wise-safe controller measurable under O , and suppose it allows some $e \in E$. If $O(e) \in O(D)$, then there is a dangerous $e' \in D$ with $O(e') = O(e)$. Measurability forces $K_O(e') = K_O(e) = \text{allow}$, contradicting safety. Hence $O(e) \notin O(D)$, and the definition of $K_O^\star$ gives $K_O^\star(e) = \text{allow}$. Therefore no O -measurable safe controller allows an event that $K_O^\star$ denies. $\square$

Corollary (Restatement of Corollary 2: Observation collisions). $K_O^\star$ rejects no benign event exactly when $\text{Collision}(O) = O(D) \cap O(E \setminus D) = \emptyset$.

PROOF. ($\implies$) Assume $K_O^\star$ rejects no benign event. For any $e \in E \setminus D$: $K_O^\star(e) = \text{allow}$, so $O(e) \notin O(D)$. Since e was arbitrary, $O(E \setminus D) \cap O(D) = \emptyset$.

($\impliedby$) Assume $\text{Collision}(O) = \emptyset$. For any benign $e \in E \setminus D$: $O(e) \in O(E \setminus D)$, and since $O(E \setminus D) \cap O(D) = \emptyset$, $O(e) \notin O(D)$, so $K_O^\star(e) = \text{allow}$. $\square$

Why event-taint observation is sufficient. Corollary 2 gives a general criterion: an observation avoids forced benign denial precisely when it does not merge dangerous and benign events. The next proposition instantiates that criterion for SAGE. By Lemma 1, one-step danger depends only on an event's propagated taint and its forbidden write-label set. When (a, t, p) resolves that write set, O_{event} retains exactly this decision-relevant information.

Proposition 1 (Sufficiency of event-taint observation). Suppose (a, t, p) uniquely resolves $\text{Wr}(u)$ and hence $Q_B(u)$. Then $\text{Collision}(O_{\text{event}}) = \emptyset$.

PROOF. By Corollary 2, it suffices to rule out an observation class containing both a dangerous and a benign event. We therefore take two events with the same event-taint observation, show that the observation fixes both inputs to Lemma 1's danger predicate, and conclude that the two events have the same danger status.

Suppose $O_{\text{event}}(e_1) = O_{\text{event}}(e_2)$ for $e_1 = (x_1, u_1), e_2 = (x_2, u_2) \in E$. Then $(a_1, t_1, p_1, \eta_{u_1}(x_1)) = (a_2, t_2, p_2, \eta_{u_2}(x_2))$. So $a_1 = a_2, t_1 = t_2, p_1 = p_2$, and $\eta_{u_1}(x_1) = \eta_{u_2}(x_2)$.

By hypothesis, (a, t, p) uniquely resolves $\text{Wr}(u)$, so u_1 and u_2 have the same write targets: $\text{Wr}(u_1) = \text{Wr}(u_2)$, hence $Q_B(u_1) = Q_B(u_2)$.

By Lemma 1: $e \in D \Leftrightarrow \eta_u(x) \cap Q_B(u) \neq \emptyset$. Since $\eta_{u_1}(x_1) = \eta_{u_2}(x_2)$ and $Q_B(u_1) = Q_B(u_2)$:

$$\begin{aligned} e_1 \in D &\Leftrightarrow \eta_{u_1}(x_1) \cap Q_B(u_1) \neq \emptyset \\ &\Leftrightarrow \eta_{u_2}(x_2) \cap Q_B(u_2) \neq \emptyset \\ &\Leftrightarrow e_2 \in D. \end{aligned}$$

Therefore $e_1 \in D \Leftrightarrow e_2 \in D$: no observation class straddles the dangerous/benign boundary. Hence $\text{Collision}(O_{\text{event}}) = \emptyset$. $\square$

Proposition 2 (ObservationSAGE exactness and complete template-library certificate). Assume complete enumeration of a finite event domain E , a finite observation O , and a finite O -measurable template library Γ , where each template denotes the observations it rejects, $\llbracket Y \rrbracket_O \subseteq O(E)$. A template

is canonical-compatible when it rejects only dangerous observations; let Γ_{compat} contain exactly these templates:

$$\Gamma_{\text{compat}} = \{\gamma \in \Gamma \mid \llbracket \gamma \rrbracket_O \subseteq O(D)\}.$$

Let W be the dangerous observations left uncovered even after admitting every canonical-compatible template:

$$W = O(D) \setminus \bigcup_{\gamma \in \Gamma_{\text{compat}}} \llbracket \gamma \rrbracket_O.$$

OBSERVATIONSAGE returns a guard set C inducing $K_O^\star$ iff $\bigcup_{\gamma \in \Gamma_{\text{compat}}} \llbracket \gamma \rrbracket_O = O(D)$. Otherwise W is nonempty and certifies that no $C \subseteq \Gamma$ induces $K_O^\star$. If enumeration is incomplete, the algorithm returns **UNKNOWN**, not either conclusion.

PROOF. The desired controller denies exactly $O(D)$. There are two exhaustive cases: every dangerous observation is covered by canonical-compatible templates, in which case selecting them realizes the controller; or some dangerous observation remains uncovered, in which case that observation is the required impossibility certificate.

Coverage. Suppose $W = \emptyset$. Equivalently, the union of the canonical-compatible templates is exactly $O(D)$. **SELECTCOVER** returns a subset $C \subseteq \Gamma_{\text{compat}}$ with that same union. Every selected template rejects no observation outside $O(D)$, and together they reject every observation in $O(D)$. The induced controller therefore denies exactly $O(D)$, which is $K_O^\star$.

Certificate. Suppose $W \neq \emptyset$ and choose $w \in W$. Any $C \subseteq \Gamma$ inducing $K_O^\star$ may contain only templates from Γ_{compat} : including a template that rejects an observation outside $O(D)$ would make the union deny a benign observation. But w is covered by no template in Γ_{compat} , hence by no such C , contradicting that $K_O^\star$ denies every observation in $O(D)$. Thus W is a complete certificate relative to Γ . The argument relies on the complete set $O(D)$; bounded enumeration therefore justifies only **UNKNOWN**.

These cases establish both outcomes claimed by the proposition: an exact guard set when coverage exists, and a complete relative certificate otherwise. $\square$

Compilation invariant. Let $\widehat{C}_B$ denote the clause set immediately before **SemanticNormalize**. The grouping phase partitions every resolved action with $Q_B(u) \neq \emptyset$ by $(a, t, Q_B(u))$. Hence each such action $u = (a, t, p)$ belongs to one group whose exact parameter filter matches p , and that group's emitted clause has label set exactly $Q_B(u)$. Conversely, an emitted pre-normalization clause whose agent, tool, and parameter filter match u is the clause for this same group and has that same label set. Finally, semantic normalization preserves Deny by its contract in the main paper. This invariant is the bridge between the model-level set $Q_B(u)$ and a deployed clause label C_c ; it is a consequence of the **DIRECTSAGE** algorithm, not an additional correctness assumption.

Theorem (Restatement of Theorem 3: Direct synthesis correctness). *Let C_B be the output of **DIRECTSAGE**. If each parameter class preserves $Q_B(u)$ and clauses use the trigger semantics of the main paper, then for every safe state x and enabled action u , $\text{Deny}_{C_B}(x, u) \Leftrightarrow \text{Deny}_B^\star(x, u)$.*

PROOF. We prove that the compiler realizes the exact model-level predicate, then apply the normalization contract. Let $\widehat{C}_B$ be the pre-normalization clauses. By the compilation invariant above, every action with a nonempty $Q_B(u)$ has exactly one matching clause in $\widehat{C}_B$, whose label set is $Q_B(u)$; no matching clause carries a different label set. Since semantic normalization preserves Deny, it suffices to prove $\text{Deny}_{\widehat{C}_B}(x, u) \Leftrightarrow \text{Deny}_B^\star(x, u)$.

($\Rightarrow$) Assume $\text{Deny}_{\widehat{C}_B}(x, u)$. Some clause $c = (a_c, t_c, pf_c, C_c)$ triggers: $u = (a', t', p')$ with $a' = a_c$, $t' = t_c$, $\text{Match}(p', pf_c)$, and $\eta_u(x) \cap C_c \neq \emptyset$. The compilation invariant identifies this matching clause as u 's group clause, so $C_c = Q_B(u)$. Therefore $\eta_u(x) \cap Q_B(u) \neq \emptyset$, i.e., $\text{Deny}_B^*(x, u)$.

($\Leftarrow$) Assume $\text{Deny}_B^*(x, u)$: $\eta_u(x) \cap Q_B(u) \neq \emptyset$. First, $Q_B(u) \neq \emptyset$ (otherwise the intersection is empty). The compilation invariant gives a matching clause in $\widehat{C}_B$ with label set $Q_B(u)$. This clause triggers on (x, u) : agent and tool match, parameter filter matches p , and $\eta_u(x) \cap Q_B(u) \neq \emptyset$. Therefore $\text{Deny}_{\widehat{C}_B}(x, u)$, and normalization yields $\text{Deny}_{C_B}(x, u)$.

Thus compilation realizes Deny_B^* before normalization, and the normalization contract preserves that equality for the deployed guard set. $\square$

Theorem (Restatement of Theorem 4: Termination and safety of additive repair). *Assume $\text{Safe}_B(x_0)$, finite Ω , complete verification, and an additive O -measurable template language that can block every returned dangerous observation. CEG-REPAIR performs at most $|O(D)|$ effective additions. If it returns C after PASS, then $\text{TraceSafe}(K_C, x_0, B)$.*

PROOF. The theorem has two independent conclusions, so we use two different arguments. For termination, finiteness of $O(D)$ supplies a ranking set: every nonterminal repair iteration must remove at least one of its members. For safety, no induction over repair rounds is needed: a PASS verdict is already a complete-reachability certificate for the final guard.

Termination: a decreasing ranking set. Let $A_i \subseteq O(D)$ be the dangerous observations still allowed after iteration i . If iteration i does not return PASS, UNKNOWN, or UNREALIZABLE, complete verification has returned a dangerous final observation $o \in A_i$. The covering premise provides an O -measurable template that blocks o , and the repair step adds such a template. Clause addition and semantic normalization cannot turn an existing denial into an allowance, so $A_{i+1} \subseteq A_i$; moreover, $o \notin A_{i+1}$ and therefore $A_{i+1} \subsetneq A_i$. Every effective addition therefore consumes a distinct member of the finite set $O(D)$. A run can make at most $|O(D)|$ effective additions; it may of course return PASS earlier when no dangerous event is reachable under its current guard. If no template covers a returned observation, the algorithm instead returns UNREALIZABLE; an incomplete search returns UNKNOWN.

Safety: interpreting a PASS verdict. Suppose the algorithm returns C after PASS. By the verifier's contract, complete BFS has exhausted the controlled reachable space from x_0 without reaching a state that violates Safe_B . By Definition 5, this is exactly $\text{TraceSafe}(K_C, x_0, B)$.

Together, finite descent bounds the number of effective additions, while a successful final verification establishes the claimed trace safety. $\square$

Proposition 3 (Persistence of inherited over-enforcement). *The preceding termination and trace-safety result is safety-only: an additive repair cannot remove benign denials already present in its input. If repair only adds clauses, every event denied by C_0 remains denied by the repaired C .*

PROOF. At iteration i , repair computes $C_{i+1} = \text{Normalize}(C_i \cup \{y_i\})$. Adding a clause cannot make an existing trigger false, so for every event e ,

$$\text{Deny}_{C_i}(e) \Rightarrow \text{Deny}_{C_i \cup \{y_i\}}(e).$$

Normalization preserves Deny; hence $\text{Deny}_{C_i}(e) \Rightarrow \text{Deny}_{C_{i+1}}(e)$. Applying this implication inductively from $i = 0$ through the final iteration gives $\text{Deny}_{C_0}(e) \Rightarrow \text{Deny}_C(e)$. Thus an initial benign denial cannot be repaired away by an additive-only run. $\square$

3 Concrete Execution Soundness

This section formalizes the forward-simulation obligation used by the deployment-soundness result in the main paper.

Definition 1 (Concrete execution soundness, restated). Let S be the concrete-state set and $I(s, v) \subseteq L$ the modeled source influences on location v in concrete state s . Define the overapproximation relation:

$$s \sqsubseteq_I x \Leftrightarrow \forall v \in V : I(s, v) \subseteq x(v).$$

The concrete tool implementation is *soundly simulated* by M if for every mediated concrete step $s \xrightarrow{u}_{\text{conc}} s'$ and every $s \sqsubseteq_I x$:

$$s' \sqsubseteq_I F_u(x).$$

The simulation obligation is a *human-confirmed premise*: static wrapper analysis, instrumentation, provenance logs, and review provide evidence, but SAGE does not automatically certify it. If the obligation is violated, the deployment theorem no longer applies: an omitted concrete influence can produce a false negative. Independently, spurious modeled influences can cause additional benign denial.

4 Deployment-Scale Domains and Model Acquisition

This section makes explicit two practical boundaries behind the main paper’s finite-model premise: how a deployment can bound a verification domain, and which parts of its model can be acquired automatically. These are engineering guidelines and deployment obligations, not additional soundness theorems.

4.1 SAGE Domains as a Scalability Boundary

Explicit-state verification has an exponential worst-case state space: a complete exploration distinguishes the reachable combinations that can affect the safety decision. Symbolic methods can prove some safe instances without enumerating them, as the PDR feasibility study below illustrates.

The intended deployment mitigation is a *logical SAGE domain*: a bounded tool cluster whose tools, shared resources, and relevant labels jointly define one policy-complete verification problem. We write a domain as $\mathcal{D}_i = (V_i, U_i, L_i, B_i, \Pi_i)$, where Π_i is a set of explicitly declared boundary ports. A local exploration then depends on $|V_i|$, $|U_i|$, and $|L_i|$, rather than on the full deployment inventory. Domains should be drawn around the tool clusters that actually share resources or exchange policy-relevant information, not around arbitrary process or network boundaries.

Ownership and handoff. A partition is valid only when it makes every policy-relevant flow explicit. Each resource carrying taint state has one owning domain. A cross-domain effect is modeled as a declared handoff: the source domain emits an output event with an overapproximation of every label that may cross the handoff, and the destination domain receives the corresponding input event before its physical effect commits. The handoff’s destination binding, policy, and label summary are reviewed semantic inputs: the deployment owner or designated security reviewer confirms them, while SAGE checks their consistency rather than inferring them.

For example, if a file produced in domain A can be read by an external-send tool in domain B , A exports the file’s possible labels through a declared handoff and B incorporates them before authorizing the send. A direct shared file path with no such handoff is not a valid domain boundary.

Scope of the composition claim. Under these conditions, local verification results can be composed, and each domain may be explored separately. If shared state has no owner, a handoff bypasses its mediator, or a label transfer is omitted, the decomposition no longer supports a global claim; the

affected components must instead be modeled together as one larger domain, or supplied with the missing reviewed handoff contract. Domains reduce the verification problem when policy-relevant state can be partitioned through explicit handoffs; otherwise, the affected components must be verified together.

Enforcement placement. The prototype places the domain enforcement point in a sidecar, but the theory requires only one complete pre-commit mediator with the resolved read/write bindings, durable-state discipline, and physical execution isolation that preserves the authorized read state. The same role can be implemented in a gateway or an embedded library. A static binary allowlist is useful for inventory and admission control, but it does not provide mediation by itself: the deployment still needs an invocation, capability, or network boundary that prevents listed programs from bypassing SAGE and rejects unlisted ones.

4.2 What Can Be Automated in Model Acquisition

SAGE distinguishes deployment structure from security meaning. Automation can show which components, endpoints, and bindings are present in a deployment; it cannot by itself establish what those operations read, write, or propagate. Thus a discovered inventory is useful evidence about model freshness, not evidence that the security model is semantically complete.

Syntax acquisition. Automation can collect and cross-check structural facts: component identities and build digests; endpoint and tool names; parameter schemas; deployment locations; declared dependencies; and the bindings by which an invocation resolves a concrete resource. A unified API server is a convenient collection point when every tool call passes through it. An agent harness provides the same benefit only when it centrally mediates all tool invocations; many agent architectures do not have such an entry point. In distributed deployments, an SDK for owned components can emit the same versioned manifest and resolved-binding events. These mechanisms lower the cost of keeping the *topology* fresh and allow SAGE to reject a semantic model whose declared inventory no longer matches the deployed generation.

Semantic confirmation. The model still needs security meaning: whether an operation reads or writes a resource, which labels it generates or propagates, which destinations are forbidden, and whether a sanitizer or declassifier is justified. These are semantic properties of arbitrary programs and business data; syntactic discovery cannot in general confirm them soundly. An LLM may draft this contract from code, documentation, traces, and domain prompts, but remains an untrusted author. Review, targeted tests, provenance evidence, and model revision are therefore part of the deployment contract. Counterexample-guided repair can repair a *guard* relative to the current model; it cannot prove that a missing real-world effect was present in that model.

Opaque components. For a precompiled or third-party binary, SAGE cannot infer a complete semantic contract merely by locating the binary. The deployer must provide a reviewed manifest for the permitted tool surface and arrange complete mediation of that surface. A directory or mounted-volume allowlist can make the candidate set finite, while build digests can detect replacement, but neither establishes read/write semantics nor blocks an unmediated network path by itself. A binary for which these conditions cannot be established is outside the confirmed SAGE domain and must be isolated or rejected at the boundary rather than silently treated as modeled.

This division identifies the realistic automation target: automatically maintain a trustworthy syntactic inventory, use LLMs to reduce the cost of drafting semantic contracts, and require explicit confirmation before SAGE attaches its conditional safety guarantee.

4.3 State-Preserving Policy Updates

The preceding discussion distinguishes a policy update from a model update. For a fixed confirmed model M and state space X , a change from B_0 to B_1 admits a staged replacement: compile the candidate controller K_{C_1} off the request path, validate the compilation for B_1 , and atomically replace K_{C_0} by K_{C_1} at the pre-commit boundary. The replacement changes the decision function, not the information-flow state:

$$x^- = x^+, \quad \text{decision}(x, u) = \begin{cases} K_{C_0}(x, u), & \text{before the replacement linearization point,} \\ K_{C_1}(x, u), & \text{after the replacement linearization point.} \end{cases}$$

In particular, taint accumulated before the update remains an input to the new controller. Discarding it would incorrectly “launder” historical data when a policy is tightened. A serialized boundary must therefore make each authorize–execute operation and each controller replacement indivisible: an invocation is evaluated wholly by one controller version.

Model changes. The state-preserving policy replacement fixes M . When a reviewed tool contract, read/write/*Gen* behavior, or label interpretation changes, the affected model facts must be re-confirmed and a new controller compiled and verified. Carrying accumulated state across an incompatible model change additionally needs an explicit compatibility argument.

5 Extended Evaluation: PDR Feasibility

While the main text focuses on DIRECTSAGE and explicit BFS, we additionally tested whether a property-directed symbolic backend can alleviate the state-space explosion boundary for safe external candidates. We encoded the finite union-flow semantics as constrained Horn clauses and used Z3 Spacer. This is a backend feasibility study, not a new PDR algorithm.

Table 1 reports the feasibility study. PDR does not replace BFS for counterexamples (on unsafe scenarios, BFS finds witnesses faster), but it converts safe instances that exhaust explicit search into inductive proofs.

Table 1. Safe-candidate backend feasibility study. BFS uses a 100,000-state limit; times are three-repetition medians.

Family	Bits	Act.	BFS	PDR
$n = 8$	80	10	PASS 0.046 s	PASS 0.056 s
$n = 12$	168	14	PASS 1.414 s	PASS 0.130 s
$n = 16$	288	18	PASS 36.589 s	PASS 0.229 s
$n = 18$	360	20	UNK. 30.402 s	PASS 0.345 s
s7 choke	95	17	UNK. 39.040 s	PASS 0.216 s

6 Complete Algorithm Pseudocode

Algorithm 2 OBSERVATIONSAGE(M, B, x_0, O, Γ)

Require: M, B, x_0, O , and Γ
Ensure: controller C , exact witness, or UNKNOWN
1: $(status, E) \leftarrow \text{ENUMERATEEVENTS}(M, x_0)$
2: **if** $status \neq \text{COMPLETE}$ **then**
3: **return** UNKNOWN
4: **end if**
5: $D \leftarrow \{e \in E \mid \neg \text{Safe}_B(F_u(x))\}$
6: $D_\Omega \leftarrow O(D)$
7: $\Gamma_{\text{compat}} \leftarrow \{\gamma \in \Gamma \mid \llbracket \gamma \rrbracket_O \subseteq D_\Omega\}$
8: **if** $\bigcup_{\gamma \in \Gamma_{\text{compat}}} \llbracket \gamma \rrbracket_O = D_\Omega$ **then**
9: **return** SELECTCOVER($\Gamma_{\text{compat}}, D_\Omega$)
10: **else**
11: $\text{uncovered} \leftarrow D_\Omega \setminus \bigcup_{\gamma \in \Gamma_{\text{compat}}} \llbracket \gamma \rrbracket_O$
12: **return** UNREALIZABLE-EXACT, uncovered
13: **end if**

Algorithm 3 CEG-REPAIR($M, B, x_0, C_0, O, \Gamma$)

1: $C \leftarrow \text{Normalize}(C_0)$
2: **loop**
3: $r \leftarrow \text{VerifyCandidate}(M, B, x_0, C)$
4: **if** $r = \text{PASS}$ **then**
5: **return** C
6: **end if**
7: **if** $r = \text{UNKNOWN}$ **then**
8: **return** UNKNOWN
9: **end if**
10: $e \leftarrow$ final event in counterexample r ; choose $\gamma \in \Gamma$ covering $O(e)$
11: **if** no such γ exists **then**
12: **return** UNREALIZABLE
13: **end if**
14: $C \leftarrow \text{Normalize}(C \cup \{\gamma\})$
15: **end loop**

Algorithm 4 VERIFYCANDIDATE(M, B, x_0, C)

Require: model M , policy B , initial state x_0 , candidate C , state budget $budget$
Ensure: PASS, FAIL (with trace), or UNKNOWN
 1: **if** $\neg \text{Safe}_B(x_0)$ **then**
 2: **return** FAIL, empty action trace at x_0
 3: **end if**
 4: $Q \leftarrow \{x_0\}$; $visited \leftarrow \{x_0\}$
 5: **while** $Q \neq \emptyset$ and $|visited| \leq budget$ **do**
 6: $x \leftarrow Q.\text{dequeue}()$
 7: **for all** u enabled at x **do**
 8: **if** $\text{Deny}_C(x, u)$ **then**
 9: **continue**
 10: **end if**
 11: $x' \leftarrow F_u(x)$
 12: **if** $\neg \text{Safe}_B(x')$ **then**
 13: **return** FAIL, trace to x'
 14: **end if**
 15: **if** $x' \notin visited$ **then**
 16: $visited \leftarrow visited \cup \{x'\}$
 17: $Q.\text{enqueue}(x')$
 18: **end if**
 19: **end for**
 20: **end while**
 21: **if** $Q = \emptyset$ **then**
 22: **return** PASS
 23: **else**
 24: **return** UNKNOWN
 25: **end if**

7 Evaluation Protocols and Detailed Results

7.1 Related-Work Scope Map

Table 2. Mechanisms and enforcement objects of systems discussed in the main paper.

System	Enforcement object	Published mechanism / authoring	Primary scope or output
FIDES	Labelled content; planner state	Dynamic IFC, label propagation, and deterministic policy enforcement	Planner expressiveness and security-utility trade-offs
CaMeL	Trusted program, untrusted data, capabilities	Trusted-flow extraction and capability-constrained execution	Prompt-injection defense under its programming model
AgentSpec	Events, predicates, enforcement actions	Trigger-predicate DSL; LLM-generated rules	Customizable runtime safety constraints
FORGE	Causal-history predicates	Datalog policy, observability contract, reference monitor	History-sensitive multi-agent enforcement
Progent	Tool calls and privileges	Privilege DSL; LLM-generated, dynamically updated policies	Least privilege and policy fallbacks
SAGE	Finite flow model; external guard	Event taint; direct compilation or verified repair	Guard validation, counterexamples, collisions, deployment proof

7.2 RQ1: Authoring and Runtime-Adaptation Details

Matched cross-interface protocol. The AgentSpec arm and FIDES tutorial-interface reproduction use the same 72 task identifiers, MiMo-v2.5 settings, semantic public inputs, fixed target probes, and oracles. AgentSpec receives a memoryless custom-predicate interface. FIDES receives its public labelled-trace policy interface and the canonical action name required by that interface. We reconstruct the FIDES policy classes and powerset lattice from the public tutorial; this is an interface-level reproduction, not the paper’s full AgentDojo runtime. In both combined arms, the verified SAGE decision runs first and the native candidate runs only if SAGE allows. Invalid candidates and runtime errors are not treated as denials.

Table 3. Matched target-probe results across independent policy interfaces. Unknown counts probes omitted from DER/BBR because the candidate was invalid or raised a runtime error.

Interface	Arm	Loaded	Err. tasks	DER	BBR	Unknown
AgentSpec	Candidate	72/72	5	61.63%	30.79%	1,608
AgentSpec	Candidate + SAGE	72/72	5	100%	30.79%	830
FIDES	Candidate	68/72	8	55.41%	30.27%	2,328
FIDES	Candidate + SAGE	68/72	8	100%	30.27%	1,379
DirectSAGE	Direct synthesis	72/72	0	100%	0.0%	0

Four FIDES candidates remain invalid after applying the same deterministic normalization used for AgentSpec outputs: two responses end at the common output limit before providing code, and two import unsupported modules. Eight loaded policies raise an exception on at least one fixed probe. These failures remain unknown in the combined arm; the reduction in unknown probes occurs when SAGE denies a dangerous target before the candidate executes. The reported DER and BBR therefore describe evaluable probes, while the loaded, error, and unknown columns expose coverage separately.

The SAGE protocol checks composition candidates using a conservative bound on reachable labels. Starting from the initial labels, we apply all unguarded union-flow transitions until reaching a fixed point. Every reachable event’s labels are contained in this bound. A candidate receives a safety certificate when its clauses cover every potentially reachable forbidden label at each action. To establish failure, we replay an admitted prefix followed by an unsafe target invocation. One MiMo candidate remains unresolved: the coverage check is inconclusive and the candidate blocks the diagnostic prefixes.

MiMo-authored predicates use AgentSpec’s custom-predicate interface and are executed by its RuleInterpreter; AgentSpec’s STOP action implements denial. The 36 composition tasks comprise six business families at six nested target dispatch counts. Within a family, larger tasks extend smaller ones, so the rows in Table 4 describe increasing compositions rather than independent samples.

Table 4. MiMo-authored AgentSpec predicates by task layer and composition size. P/F/U denotes certified safe, witnessed unsafe, and unresolved predicates. DER and BBR are percentages over evaluable fixed probes.

Task layer	P	F	U	DER	BBR
Base (36 tasks)	25	9	2	81.25%	5.56%
8 dispatches	0	5	1	58.85%	37.13%
16 dispatches	1	4	1	49.38%	24.68%
24 dispatches	0	4	2	65.48%	39.86%
32 dispatches	0	6	0	57.28%	29.95%
40 dispatches	0	4	2	64.34%	42.39%
48 dispatches	0	5	1	63.71%	21.00%
All tasks	26	37	9	61.63%	30.79%

AgentSpec guard completion. Each of the 72 predicates undergoes one model-derived completion pass followed by verification, with no additional authoring-model call. The pass removes all 37 witnessed omissions and certifies 67 predicates. Of the nine unresolved originals, five raise an error when executed by AgentSpec and therefore remain unresolved after additive completion. The other four have incomplete candidate coverage but no executable witness in the fixed diagnostic

set; adding the synthesized constraints yields a certificate. Completion retains the original 30.79% fixed-probe BBR. DirectSAGE certifies all 72 tasks with zero model-level BBR. The completion outputs use 1–12 normalized clauses and 2–46 exact action patterns, with medians of 3 and 7, respectively. All 11 integration regression tests pass.

Table 5. Two-author SAGE guard-language study on the same 72 model-policy tasks. Composition rows contain six tasks each; base contains 36. Size is the target dispatch count, with total actions in parentheses. These results assess authoring variation within the SAGE protocol of main-paper RQ1.

Task / size	MiMo-v2.5			GPT-5.5		
	P/F/U	DER	BBR	P/F/U	DER	BBR
Base (6–7 actions)	35/1/0	97.2	0.0	36/0/0	100	0.0
8 dispatches (15 actions)	5/1/0	90.0	8.0	5/1/0	93.4	6.5
16 dispatches (23 actions)	4/2/0	94.1	1.1	4/2/0	83.5	13.6
24 dispatches (31 actions)	4/1/1	93.0	2.0	3/3/0	89.7	5.7
32 dispatches (39 actions)	6/0/0	100	6.2	5/1/0	99.3	1.5
40 dispatches (47 actions)	4/2/0	97.4	0.4	4/2/0	99.1	0.8
48 dispatches (55 actions)	3/3/0	90.7	2.9	4/2/0	92.4	6.0
Initial drafts	61/10/1	94.8	2.8	61/11/0	94.1	4.4
SAGE completion	72/0/0	100	2.8	72/0/0	100	4.4
DirectSAGE	72/0/0	100	0.0	72/0/0	100	0.0

7.3 Runtime Failure Handling

Table 6. Fault-injection matrix. All scenarios matched their expected outcomes; the legacy post-commit case is an intentionally unsafe control.

Scenario	Observed	Effect?
Policy deny before backend	DENY (fail-closed)	0
Sidacar unavailable	DENY (fail-closed)	0
DTFC backend failure	DENY (fail-closed)	0
DTFC pre-commit failure	DENY (fail-closed)	0
Legacy post-commit control	ALLOW (effect occurred)	1
Ghost taint recovery (WAL)	DENY	0
WAL unwritable	DENY (fail-closed)	0
Manifest drift	DENY	0

7.4 LangGraph ToolNode Integration

Integration. We integrated SAGE with LangGraph 1.2.9 using its public synchronous and asynchronous ToolNode wrapper hooks, without modifying LangGraph.

Trusted binding and commit protocol. Agent identity is injected from trusted graph state through InjectedState, rather than exposed as an LLM-visible tool argument. The adapter uses this identity to resolve the SAGE action and does not accept caller-supplied read, write, generation, or label metadata. For an allowed call, SAGE decides and durably commits the conservative post-state before invoking the physical tool handler. A denied call returns a ToolMessage without invoking that handler. A shared runtime lock covers resolution, decision, commit, and physical completion,

Table 7. Single-host sidecar cost. Latencies are in milliseconds; goodput is in requests per second.

Mode	p50	p95	p99	Goodput
SR only	1.514	1.589	1.676	656
SR + SAGE resolution	1.514	1.584	1.691	653
SR + SAGE gRPC	4.518	5.150	6.386	220

including across adapter instances. Caller cancellation leaves an active operation holding the lane until completion. Handler errors or indeterminate termination quarantine the lane until backend reconciliation; committed ghost taint is retained. This contract requires complete mediation of conflicting accesses and no detached work after successful handler return. Independent runtimes and external writers need additional coordination.

Failure and concurrency checks. Thirteen adapter regression cases cover pre-execution commit, denied-call non-execution, ghost-taint retention, physical serialization, shared-runtime adapters, synchronous/asynchronous execution, cancellation while waiting or authorizing, active cancellation with a thread-backed handler, and failure quarantine. In a controlled late-read experiment using the real durable runtime and an in-memory backend, the archived authorization-only adapter permits a clean send to read PII written by a later call. The repaired adapter completes the clean send before authorizing that writer. This checks the specific interleaving, not arbitrary backend isolation.

This integration shows that SAGE’s trusted binding and pre-commit ordering can be realized through LangGraph’s public ToolNode boundary without modifying the framework.

7.5 Sidecar Runtime Cost

The sidecar benchmark compares the Security Runtime (SR) alone, SR with SAGE resolution, and SR with SAGE over loopback gRPC. Each of two workload profiles uses 100 requests per run and three repeats. Table 7 reports averages of the per-run latency percentiles and goodput, not pooled percentiles.

A separate decide-commit measurement has a median of 28.6 microseconds with in-memory state and 2.4 milliseconds with fsync-backed persistence. All measurements use one host, one consistency domain, and one serial client.

8 RQ2: Physical Execution Protocol

Main-paper RQ2 uses 72 controlled instances: three business domains with 24 instances each. The domains share mechanism templates and vary caller identity, persisted resources, destinations, and policies. The two-source template reads both a SQLite record and a JSON attachment. The trusted sanitizer replaces its input with a fixed safe summary.

Six monitors execute all instances from fresh logical snapshots. The audited harness reads persisted content and labels, retains before/after input and output snapshots, and checks exact output bodies, destination identities, and counts. It checks 432 executions in total. SAGE’s 33 allowed calls produce 39 effects: 30 loopback HTTP sends, six SQLite writes, and three deployment-file writes. Its 39 denied calls produce none. Tests mutate both SQLite and JSON resources after setup to establish that observation and execution use backing storage.